SBMS SYSTEM ARCHITECTURE AND MODULE MAP

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `backend/src/index.js`, `backend/src/routes/*`, `backend/src/controllers/*`, `backend/src/services/*`, `backend/src/models/*`, `backend/src/config/*`, `backend/src/middleware/*`, `frontend/src/**`, `ml-service/predict.py`, `ml-service/feature_engineering.py`, `Dockerfile`.

---

1. Architecture as built

SBMS is a three-tier web application with a fourth, in-process inference component:

```
Browser (React SPA, Vite build)
   |  HTTPS + JSON, Bearer JWT
   v
Express API (Node 22)  ->  13 routers -> 13 controllers -> 10 services -> 5 models
   |  stdio JSON lines
   +--> Python predictor (predict.py --server, joblib artifact)
   |
   v  TCP, TLS in deployment
PostgreSQL 17 (13 tables, 14 forward migrations)
```

Tier	Responsibility	Deliberately not responsible for
Presentation	Rendering, routing by role, form state, plain-language messaging	Business rules, authorisation decisions, any scoring
Application	Authentication, authorisation, validation, workflow transitions, audit, response shaping	Rendering, model training
Domain services	Screening and aggregation, review policy, eligibility arithmetic, bursary and payment rules, certificate rendering, readiness checks	HTTP concerns
Persistence	Constraints, referential actions, uniqueness, transactional integrity	Rules that need more than one row in context
Inference	Feature construction and scoring with one artifact	Deciding what a score means, creating cases, storing anything

The ML component lives inside the API container and is reached only through the Node process's stdin. There is no separate ML service, no message broker and no job queue anywhere in the system.

2. Request lifecycle and middleware order

Order	Middleware	Purpose	Failure behaviour
1	Security headers	`nosniff`, `DENY`, `no-referrer`, restricted permissions	—
2	Origin allowlist check	Refuse a disallowed browser origin before routing	403 `Origin is not allowed`
3	`cors`	Reflect allowlisted origins only	—
4	Proxy trust (conditional)	Honour `TRUST_PROXY_HOPS` when set	—
5	`POST /api/verify` chain	Rate limit, then JSON parse, then handler	429, 400 or 503
6	`POST /api/certificates/verify` chain	Same, with its own limiter	429, 400 or 503
7	Shared JSON parser	100 kB body limit for all other routes	`Request body is too large`
8	`/api/health`, `/api/ready`	Liveness and readiness	503 when not ready
9	Eleven mounted routers	Feature endpoints with their own guards	401, 403 or 4xx

Order	Middleware	Purpose	Failure behaviour
10	`errorHandler`	Sanitise every thrown error	Generic 5xx; `Invalid request` for 4xx

The order matters in three places: the public verifiers are mounted before the shared parser so an abusive client is rejected before its body is read; the origin check precedes routing so no route can be reached from a disallowed origin; the error handler is last so every controller can simply call `next(error)`.

3. Backend module map

3.1 Routers (13) and their guards

Router	Mount	Guard
`auth.routes.js`	`/api/auth`	per route: limiter for sign-in, `authenticateToken` for `/me`
`students.routes.js`	`/api/students`	router-level `authenticateToken` + `requireRole('field_officer')`
`duplicateFlags.routes.js`	`/api/duplicate-flags`	router-level `authenticateToken` + `requireRole('admin')`
`bursaryAwards.routes.js`	`/api/awards`	router-level `authenticateToken`; per-route roles
`disbursements.routes.js`	`/api/disbursements`	router-level `authenticateToken`; per-route roles
`eligibility.routes.js`	`/api/eligibility`	`authenticateToken` + `requireRole('field_officer')`
`settings.routes.js`	`/api/settings`	`authenticateToken` + `requireRole('admin')`
`adminDashboard.routes.js`	`/api/admin/dashboard`	`authenticateToken` + `requireRole('admin')`
`reports.routes.js`	`/api/reports`	`authenticateToken` + `requireRole('admin')`
`audit.routes.js`	`/api/audit-logs`	`authenticateToken` + `requireRole('admin')`
`users.routes.js`	`/api/users`	router-level `authenticateToken` + `requireRole('admin')`
`beneficiaries.routes.js`	`/api/beneficiaries`	router-level `authenticateToken`; officer for writes
inline routes	`/api/health`, `/api/ready`, `/api/verify`, `/api/certificates/verify`	none, or a dedicated limiter

3.2 Services (10) and what each owns

Service	Owns	Called from
`duplicateCheck.js`	Spawning and supervising the predictor, batching pairs, mapping failure to a 503	`students.controller.js`, `duplicateFlags.controller.js`
`duplicateAggregation.js`	`top-1` / `all` policy, candidate ranking, suppression counts, policy normalisation	`students.controller.js`
`mlDuplicateReview.js`	Exact-identifier detection, priority banding, the shared `configuredThreshold` accessor	Registration and queue
`eligibility.js`	The five-factor arithmetic and category banding	`eligibility.controller.js`
`bursaryRules.js`	Payment transitions, cents comparison, receipt status, overdue arithmetic, exception reasons	`disbursements.controller.js`
`certificateDocument.js`	PDF layout with pdfkit and the QR code	`certificates.controller.js`
`auditLog.js`	Writing an `audit_logs` row inside the caller's transaction	Every mutating controller

Service	Owns	Called from
`studentRecordFormat.js`	Identifier and phone normalisation, ISO date validation	`students.controller.js` and others
`databaseReadiness.js`	Required-table and database-identity contract	`/api/ready`, tests
`databaseStartup.js`	Start-up assertion on the login schema	`index.js`

3.3 Models (5) — thin data access

Model	Tables it touches
`student.model.js`	`students` (create, list, read; strips `qr_token` from responses)
`beneficiary.model.js`	`beneficiaries` (legacy)
`bursaryCertificate.model.js`	`bursary_certificates`
`disbursement.model.js`	`disbursements`
`audit.model.js`, `auth.model.js`	`audit_logs`, `users`

Several controllers also issue SQL directly (awards, duplicate flags, eligibility persistence), so the model layer is a convention rather than a mandatory boundary. That is a real design inconsistency: a reader should not assume every query passes through a model.

3.4 Configuration modules

Module	Responsibility	Fail-fast behaviour
`config/db.js`	Shared `pg` pool	Connection errors surface through the handler
`config/jwtSecret.js`	Validate and hold the signing secret	Throws on length, placeholder or low diversity
`config/cors.js`	Parse and validate the origin allowlist	Throws on a malformed origin
`config/publicAppUrl.js`	Resolve and validate the public frontend origin used in QR links	Throws on a non-HTTPS production origin or a LAN address
`config/schools.js`	The five approved schools	—
`config/settings.js`	The three duplicate cut-offs in process memory	Falls back to a valid default when the environment value is out of range
`config/reconciliation.js`	`ACKNOWLEDGEMENT_OVERDUE_DAYS`	Throws on a non-integer or out-of-range value

4. Frontend module map

Layer	Files	Responsibility
Shell	`main.jsx`, `App.jsx`	Mount, route table, role guards, role-based home redirect
Layout	`components/AppLayout.jsx`, `components/PublicVerificationLayout.jsx`	Authenticated shell with navigation; minimal public shell
Context	`context/AuthContext.jsx`	Session bootstrap from `GET /auth/me`, sign-in, sign-out
Hooks	`hooks/useHashToken.js`	Fragment token capture, URL sanitisation, re-verification on `hashchange`
Transport	`services/api.js`	Axios instance, base URL, bearer interceptor
Feature panel	`components/AwardCertificatePanel.jsx`	Certificate issue, read, print, revoke
Pages	14 files in `pages/`	One screen per route in the UI inventory

5. End-to-end flows across modules

Flow	Sequence
Registration and screening	`StudentRegistration` → `POST /students` → Zod validation → advisory lock → candidate select `FOR UPDATE` → `duplicateCheck.scorePairs` → `duplicateAggregation.summarizeAggregation` → `studentModel.create` → `duplicate_flags` inserts → `auditLog` → response with QR image and screening block
Duplicate adjudication	`DuplicateReview` → `GET /duplicate-flags` (masked, prioritised) → optional audited reveal → `PATCH /duplicate-flags/:id` → transaction: status update, `duplicate_review_events` insert, audit row
Award and payment	`POST /awards` → verification event → `PATCH /awards/:id/decision` (gated on verification and duplicate state) → `POST /disbursements` (gated on approval, verification, duplicate state and amount) → `PATCH /disbursements/:id/payment-status` → `PATCH /disbursements/:id/acknowledgement` → reconciliation events and audit rows
Student verification	QR image → `/verify#token=...` → `useHashToken` → `POST /api/verify` → token format check → existence query → `{found}`
Certificate	`POST /awards/:id/certificate` → snapshot row, reference, token, QR → `/verify-certificate#token=...` → `POST /api/certificates/verify` → snapshot compared with the live award → status

6. Architectural decisions worth recording in Chapter Four

Decision	Rationale	Cost
ML inside the API container	One deployable unit, no extra port, no second authentication boundary	A predictor crash affects the API process; scaling scales the model with it
Feature construction in Python only	Training and serving cannot drift	The Node side cannot score without the subprocess
Advisory lock around registration	Serialises candidate comparison so two registrations cannot interleave	Registrations queue behind one another
Snapshot table for certificates	A printed document cannot be rewritten by a later award edit	Duplicated facts to keep consistent in the verification comparison
Human decision gates at the database	`CHECK` constraints make an unverified approval impossible, not merely discouraged	Changing policy requires a schema change
`top-1` case aggregation	Stops the queue growing with registry size	Suppressed candidates are only counted, never reviewed
Thresholds in process memory	Runtime tuning without a redeploy	Values are lost on restart
Migrations never automatic	A container restart cannot change the schema	An operator must remember to apply them

7. Architectural weaknesses to report honestly

Weakness	Consequence	Recorded as
No candidate blocking	Each registration is O(N) model calls; the rescan is quadratic	FR-34
Single process, single container	In-process rate limits, thresholds and the ML subprocess all vanish on restart and are not shared across replicas	NFR-07, §6
No queue or worker	A slow registration blocks the request; there is no background processing	Design note
SQL in both controllers and models	Two ways to reach the database; no single data-access boundary	§3.3
Legacy `beneficiaries` module still mounted	Two registries with no relationship; a source of confusion for a reader	§8
Dead `auth.controller.register`	Suggests a capability that does not exist	`SBMS_API_ENDPOINT_CATALOGUE.md` §7.1
No observability	Logging destination none in the deployment and no metrics	Deployment guide §13
No API versioning	A breaking change cannot be introduced without breaking the deployed frontend	Design note

8. Dead and legacy code inventory

Item	State	What the report should say
`beneficiaries` router, model and table	Mounted and reachable; not used by any page	A legacy module retained for compatibility
`auth.controller.register` and `registerSchema`	Unreachable behind a 410 route	Dead code, not a feature
`render.yaml`	Superseded by Azure	Historical artefact
`disbursements.status`	Legacy duplicate of `payment_status`	Kept in step, not authoritative
`duplicate_flags.probability`	Legacy column name for an uncalibrated score	Documented by a column comment
Random Forest research artifact	Never loaded by the application	Research candidate only

9. Technology stack with version evidence

Layer	Technology	Version evidence
Runtime	Node.js	v22.11.0 (test report)
API framework	Express	4.x per `backend/package.json`
Validation	Zod	Per-controller schemas
Database	PostgreSQL	17.11
Driver	`pg`	Pool in `config/db.js`
Auth	`jsonwebtoken`, `bcrypt`	Cost factor 12
QR and PDF	`qrcode`, `pdfkit`	Registration and certificate rendering
Frontend	React, Vite, `react-router-dom`, `axios`	2 006 modules in the production build
ML	Python 3.11.9, scikit-learn 1.9.0, XGBoost 3.2.0, pandas 3.0.5, jellyfish 1.2.1, joblib 1.6.0	`ml-service/requirements.txt`, experiment report
Tests	Jest, `node --test`, `unittest`	Test evidence catalogue §1
Delivery	Docker, GitHub Actions, Azure Container Apps, Azure App Service	Deployment guide §2-§5

10. What this architecture does not include

No message broker, cache, search index, service mesh, API gateway, feature store, model registry, experiment tracker, or infrastructure-as-code. Deployment was performed with CLI commands recorded in the deployment log, not from a declarative template; a reader who expects `main.bicep` or a `terraform/` directory will not find one. That absence is the honest answer to "how is this deployment reproducible" — the runbook is a document, not code.